



Deep Learning

Lecture 7.1: Neural Network Optimization (II)

Today's Learning Objectives



Be able to answer:

- What are adaptive learning rate schemes for neural networks?
 - What is RMSProp?
 - What is Adam?
 - What is AdamW?
 - What is EMA (a.k.a. Polyak Averaging)?
- What are some regularization schemes for neural networks?
 - What is ensembling?
 - What is dropout?
 - What is batch normalization?
 - What is layer normalization?
 - What is group normalization?



- We have seen that NN training is prone to failure due to step size being large
- Natural idea: If the gradient is large, step size can be smaller on that **specific dimension**

- AdaGrad:
$$r_{Tk} = \sum_{i=1}^T \|G_{ik}\|^2 = r_{T-1,k} + \|G_{ik}\|^2$$

$$\alpha_k = \frac{\alpha_0}{\sqrt{r_{Tk}}}$$

Try:

$$\min_{w_1, w_2} (w_1 - 1)^2 + 100(w_2 - 1)^2$$



- AdaGrad does not work well for non-convex problems
- Remembered too much history
- In stochastic non-convex optimization, history might be not very useful

RMSProp Formulation

$$r_{Tk} = \rho r_{T-1,k} + (1 - \rho) \|G_{Tk}\|^2 \quad \text{e.g. } \rho = 0.9$$

$$\alpha_k = \frac{\alpha_0}{\sqrt{r_{Tk}}}$$



- RMSProp is an intuitive idea, but has no statistical backup
- From a statistical point of view, RMSProp creates biased estimates of the running average:

$$r_{Tk} = \rho r_{T-1,k} + (1 - \rho) \|G_{Tk}\|^2$$

$$= (1 - \rho) \sum_i^T \rho^{T-i} \|G_{ik}\|^2$$

$$\begin{aligned} \mathbb{E}(r_{Tk}) &= \mathbb{E}(\|G_k\|^2)(1 - \rho) \sum_i^T \rho^{T-i} \\ &= \mathbb{E}(\|G_k\|^2)(1 - \rho^T) \end{aligned}$$



- Robust choice of step size
- Two moment estimates:
 - Bias-correcting momentum(T superscript here is not transpose, is T-th power!), e.g. $\rho_1 = 0.9$

$$D_T = \frac{\rho_1}{1 - \rho_1^T} D_{T-1} + \frac{1 - \rho_1}{1 - \rho_1^T} G_T$$

- Bias-correcting RMS estimate (average gradient norm), e.g. $\rho_2 = 0.999$

$$r_{Tk} = \frac{\rho_2}{1 - \rho_2^T} r_{T-1,k} + \frac{1 - \rho_2}{1 - \rho_2^T} \|G_{Tk}\|^2$$

- Final update:

$$W_k = W_k - \epsilon \frac{D_{Tk}}{\sqrt{r_{Tk}}}$$

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with
 corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

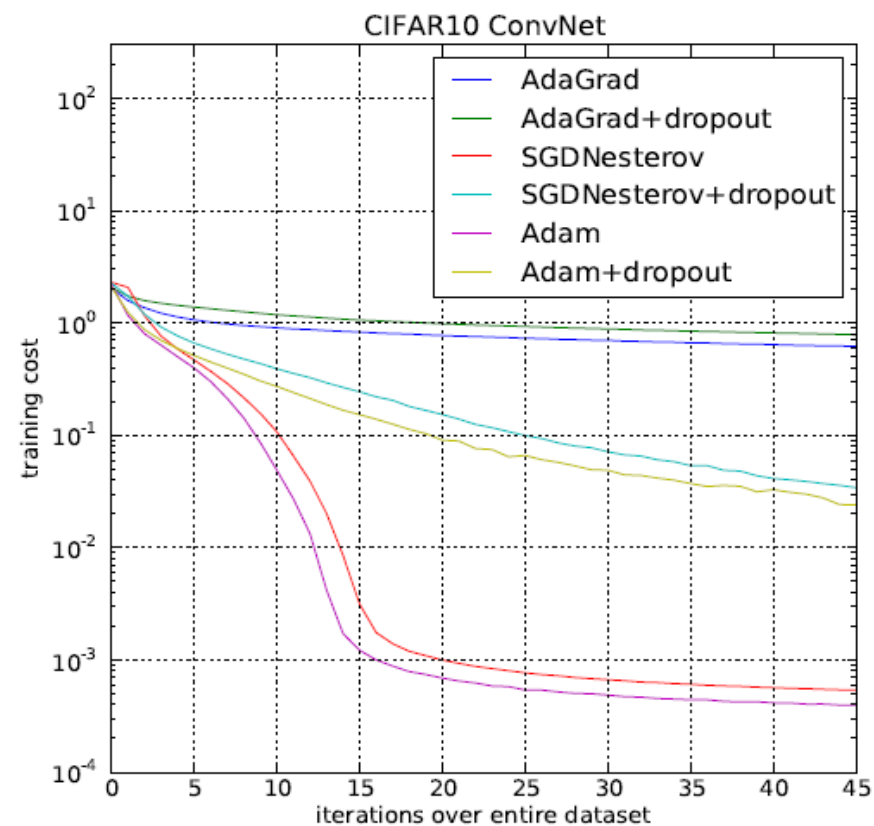
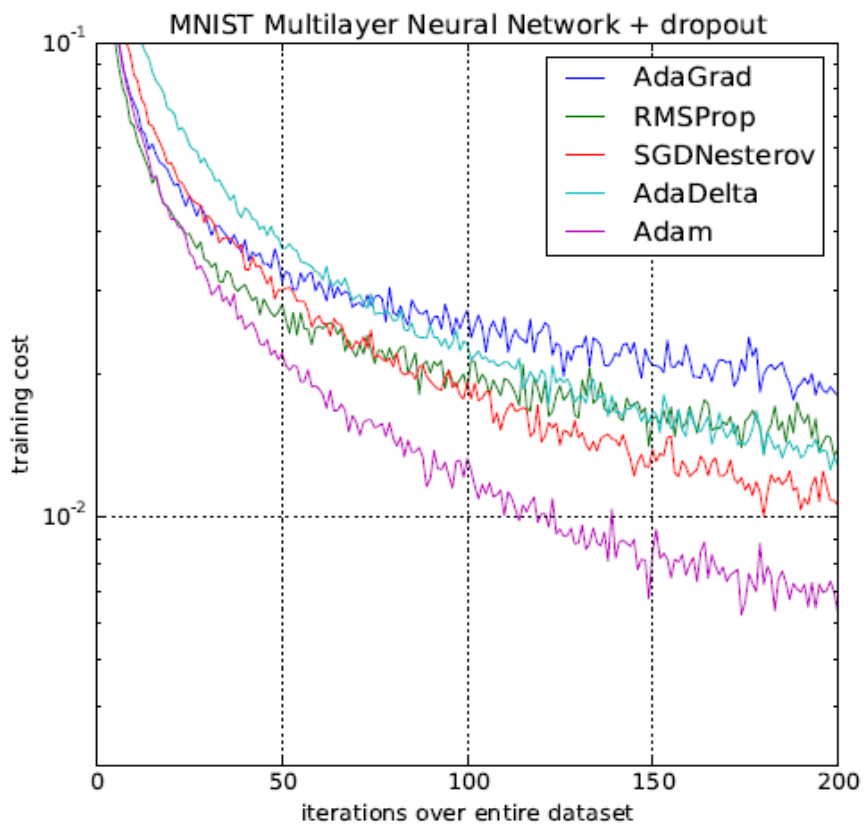
 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while





Polyak Averaging or Exponential Moving Average (EMA)

- $\widehat{W}_T = \alpha \widehat{W}_{T-1} + (1 - \alpha)W_T$
- Used **after training** for the last few checkpoints to average them for better/more stable performance during inference
- Usually can provide 0.2% - 0.4% performance gain
- EMA also used in self-supervised learning to provide a “slow-moving” teacher model

Today's Learning Objectives



Be able to answer:

- What are adaptive learning rate schemes for neural networks?
 - What is RMSProp?
 - What is Adam?
 - What is AdamW?
 - What is EMA (a.k.a. Polyak Averaging)?
- What are some regularization schemes for neural networks?
 - What is ensembling?
 - What is dropout?
 - What is batch normalization?
 - What is layer normalization?
 - What is group normalization?



- What are the best ways to prevent overfitting?
 - Best: Get **more (good) data!**
 - If can't get more data, use data augmentation
 - Sometimes, too big models trained on too small data can lead to overfitting
 - After done with all these, use the approaches described today



- In machine learning, we had bagging and random forests to ensemble different types of data
- In neural networks, it often suffices to use ensembles trained from random initializations
 - (not always.. we submitted a paper to ICML 2024 on doing something better)

Error %	Val Top-1	Val Top-5	Test Top-5
Krizhevsky <i>et al.</i> [67], 1 convnet	40.7	18.2	—
Krizhevsky <i>et al.</i> [67], 5 convnets	38.1	16.4	16.4
Krizhevsky <i>et al.</i> [67], 1 convnets*	39.0	16.6	—
Krizhevsky <i>et al.</i> [67], 7 convnets*	36.7	15.4	15.3
Our replication of Krizhevsky <i>et al.</i> [67], 1 convnet	40.5	18.1	—
1 convnet as per Figure 7.2	38.3	16.4	16.5
5 convnets as per Figure 7.2	36.6	15.3	15.3

Table 7.2: ImageNet 2012 classification error rates. The * indicates models that were trained on both ImageNet 2011 and 2012 training sets with an additional convolution layer.



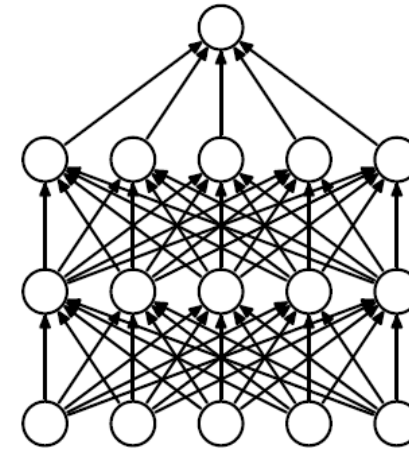
- Create multiple copies of similar testing data and average the results
- e.g. Resize the image into different sizes/aspect ratios, crop squares at different places of the image
- Early results on ensembles on multi-crops on a 2014 Google Inception network for image classification:

Number of models	Number of Crops	Cost	Top-5 error	compared to base
1	1	1	10.07%	base
1	10	10	9.15%	-0.92%
1	144	144	7.89%	-2.18%
7	1	7	8.09%	-1.98%
7	10	70	7.62%	-2.45%
7	144	1008	6.67%	-3.45%

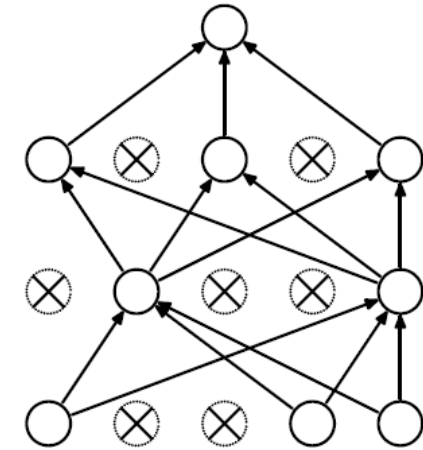


Dropout

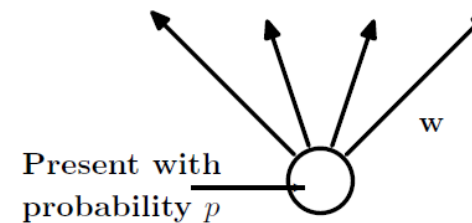
- Each time we present a training example, we randomly omit each hidden unit with probability p .
- Hinton's intuition:
 - Dropout avoids “complex conspiracies” from the neurons and asks for more individual responsibility on each neuron
- Dropout can also be considered as model averaging of all the models
- During test time, don't dropout, but use $p\mathbf{w}$



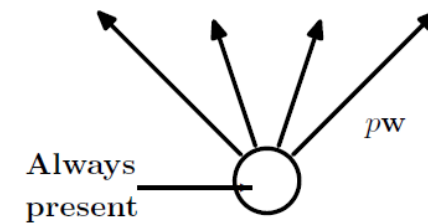
(a) Standard Neural Net



(b) After applying dropout.



(a) At training time



(b) At test time



How useful is dropout?

- Very useful in the early stages of deep learning
- Later, once normalization approaches started to become popular, people found dropout to be less and less useful
- Nowadays, few if any people employ dropout in the networks, but occasionally they still help



- Idea: Deep layers can have increased bias and magnitude
 - E.g. very deep layers activations can go to 0, or infinity
- Might be nice if every layer's output is normalized to 0 mean and 1 standard deviation!
- How to perform this normalization in a mini-batch?

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$



$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

- What if the estimate is wrong?
- Learn “correction” terms to correct it:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}.$$

- This also removes the need for an additional bias term!



Batch Normalization Algorithm

- Done on each dimension separately
- How many parameters?

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

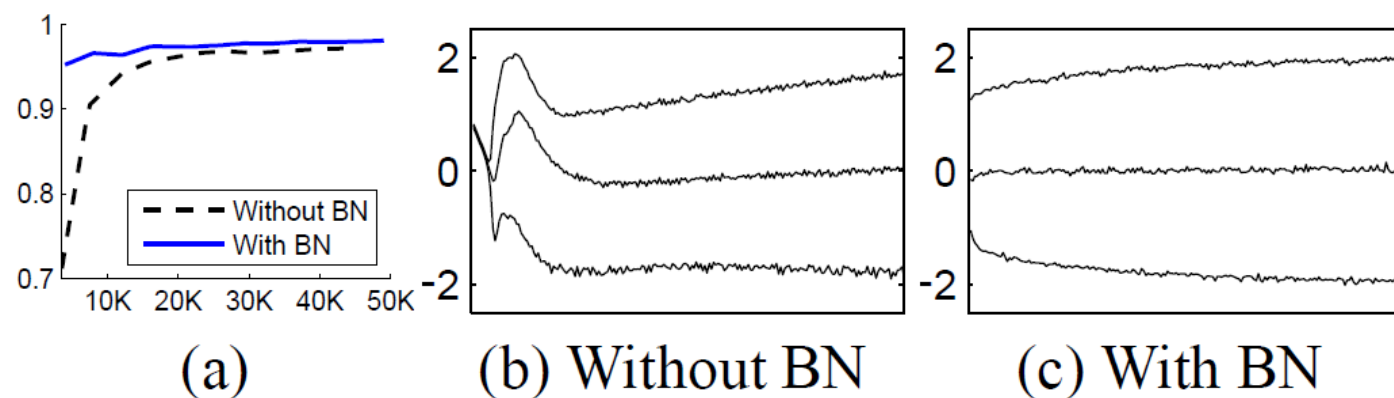


Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as {15, 50, 85}th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*



Results: Faster Learning Rate

- Once BN is employed, they were able to increase learning rate by 30x with improved results!
- Normalization alleviated the chance of gradient explosion

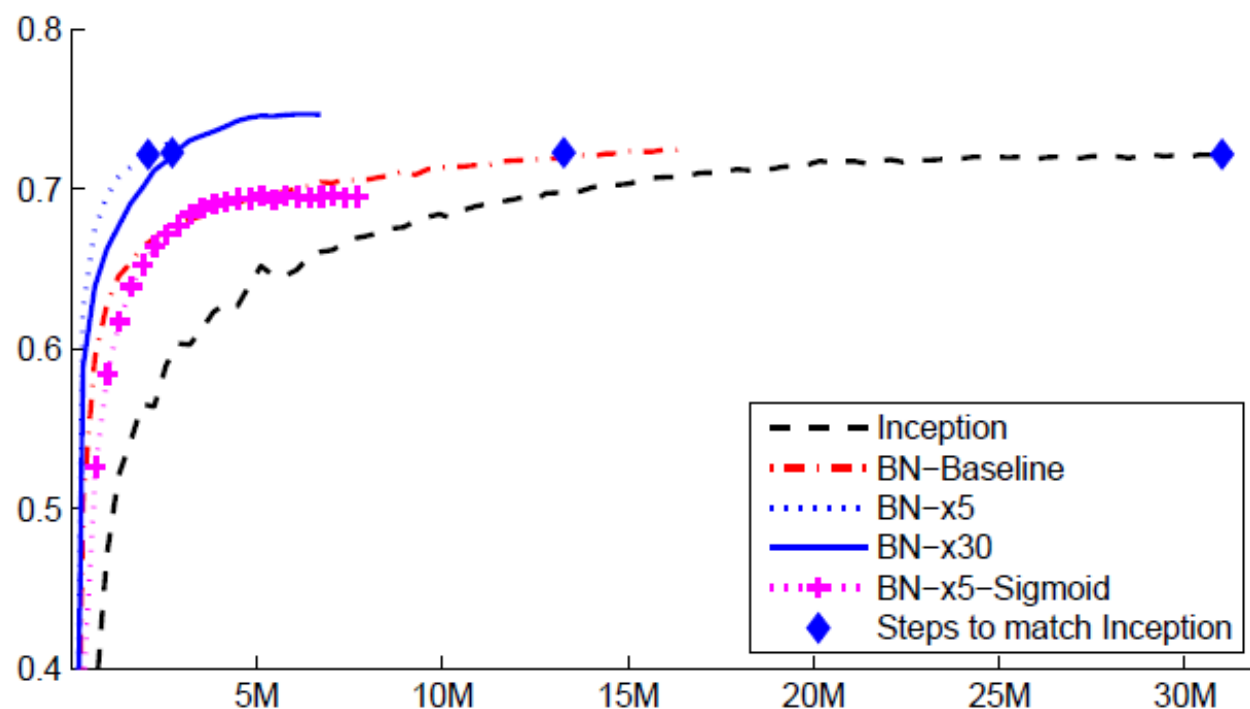


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*



Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	4.9%*

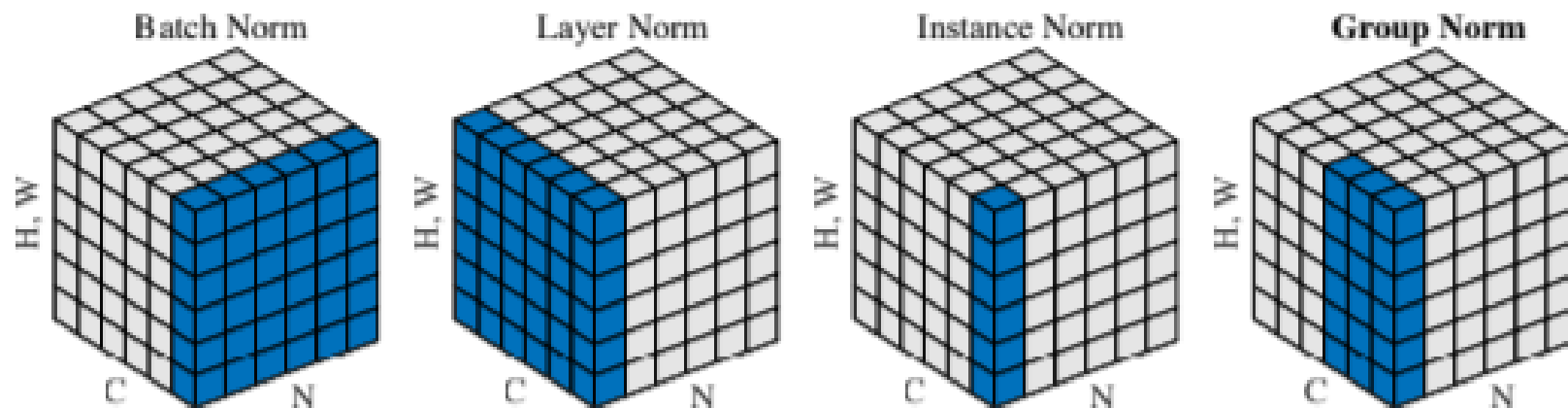
- The general formula below can be applied to different normalization schemes

$$\hat{x}_i = \frac{1}{\sqrt{\frac{1}{m} \sum_{k \in S_i} (x_k - \mu_i)^2}} (x_i - \mu_i), \mu_i = \frac{1}{m} \sum_{k \in S_i} x_k$$

- Layer normalization:
normalize over all
channels

- Usually used in
transformers

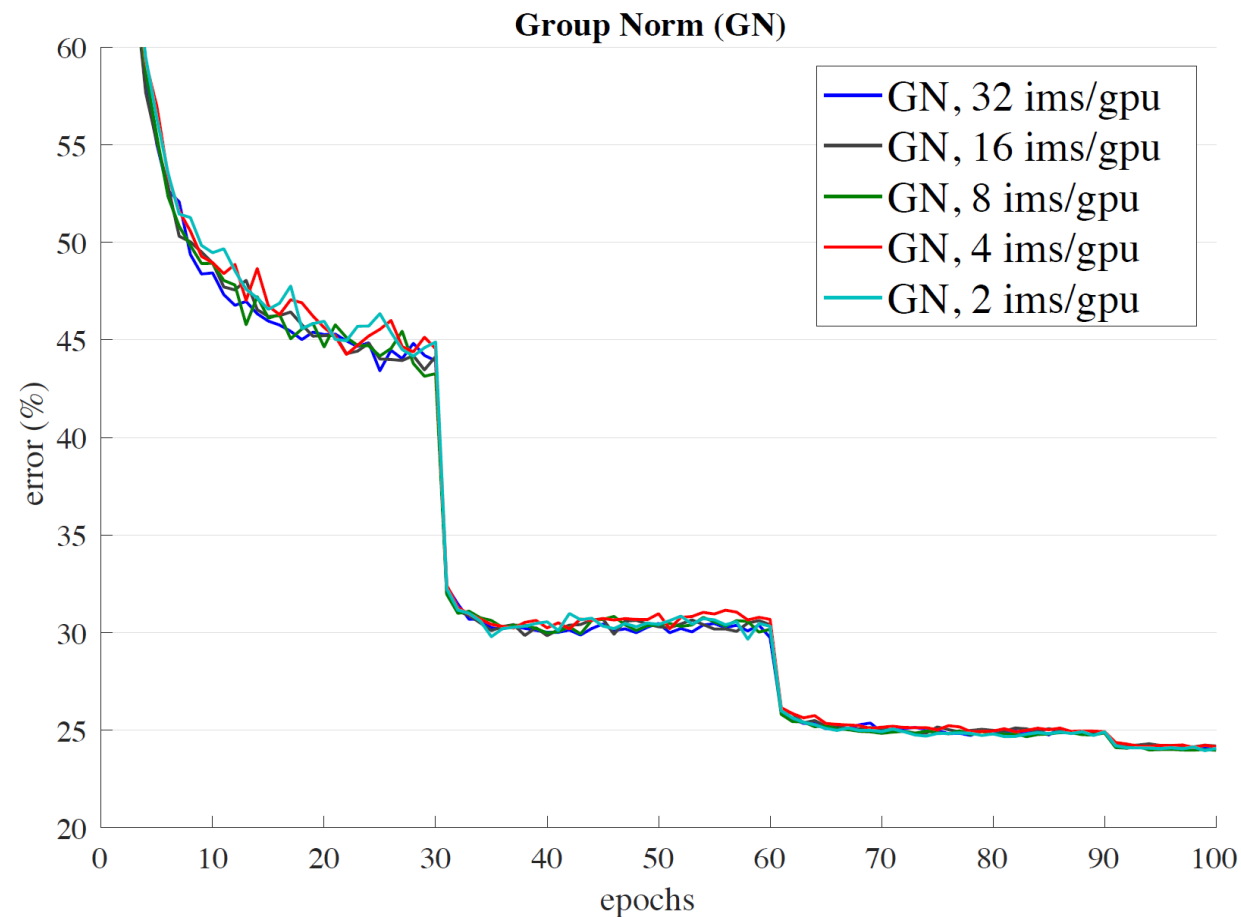
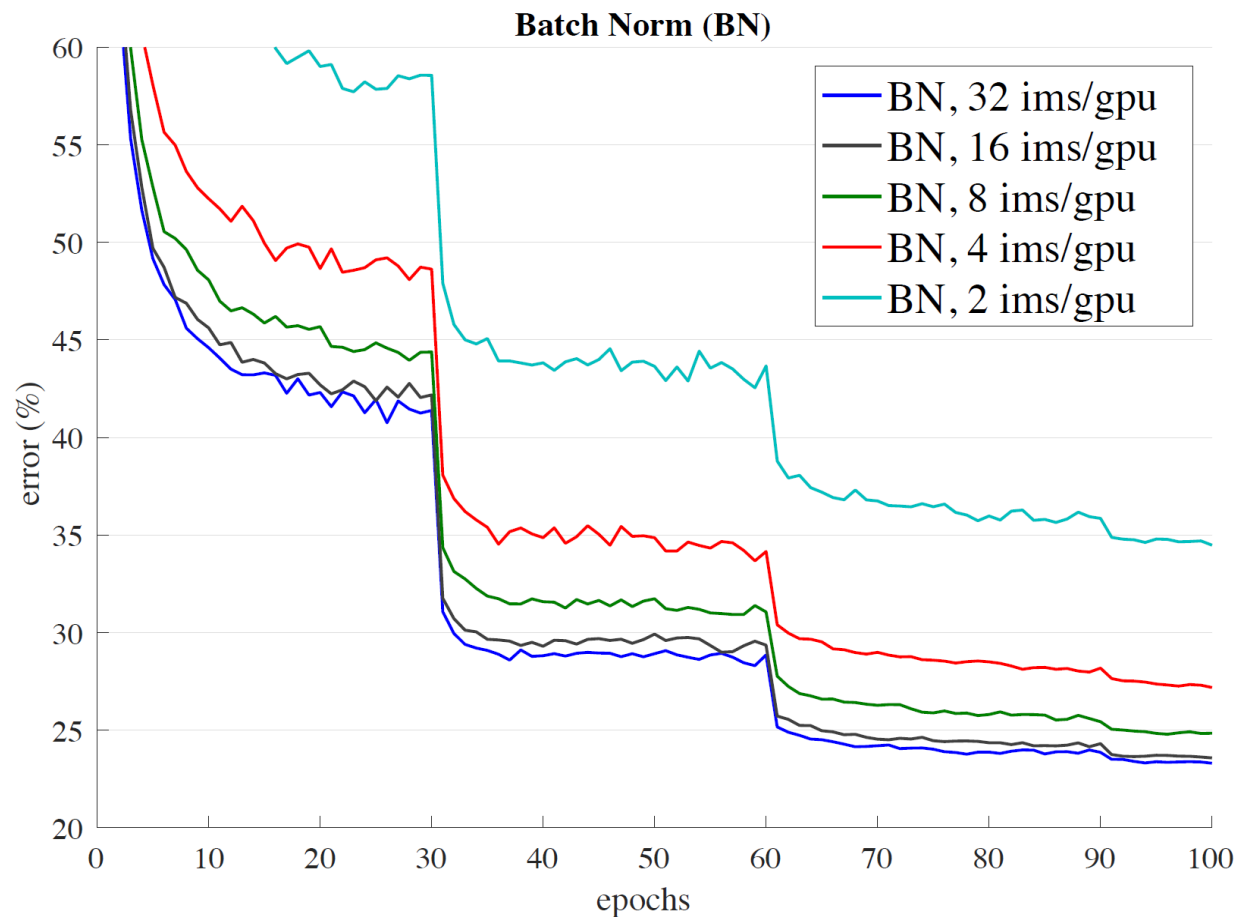
- Group normalization:
normalize over a
subset of channels





Group Normalization vs. Batch Normalization

- Batch normalization is heavily affected by batch size
- Group/layer normalization are less affected by batch size





Interesting Differences in Decision Mechanisms between Batch Normalization and Layer Normalization/Group Normalizations



Category: Harvester/Reaper

“Compositional” Behavior

(Similar to a linear or sublinear sum)

ConvNeXt, non-distilled Transformers

“Disjunctive” Behavior

(Similar to disjunctions of conjunctions)

CNNs, Distilled Transformers



Turns out: Batch Normalization leads more to “Disjunctive” behaviors
Layer/Group normalizations lead more to “Compositional” behaviors



Interesting Differences in Decision Mechanisms between Batch Normalization and Layer Normalization/Group Normalizations

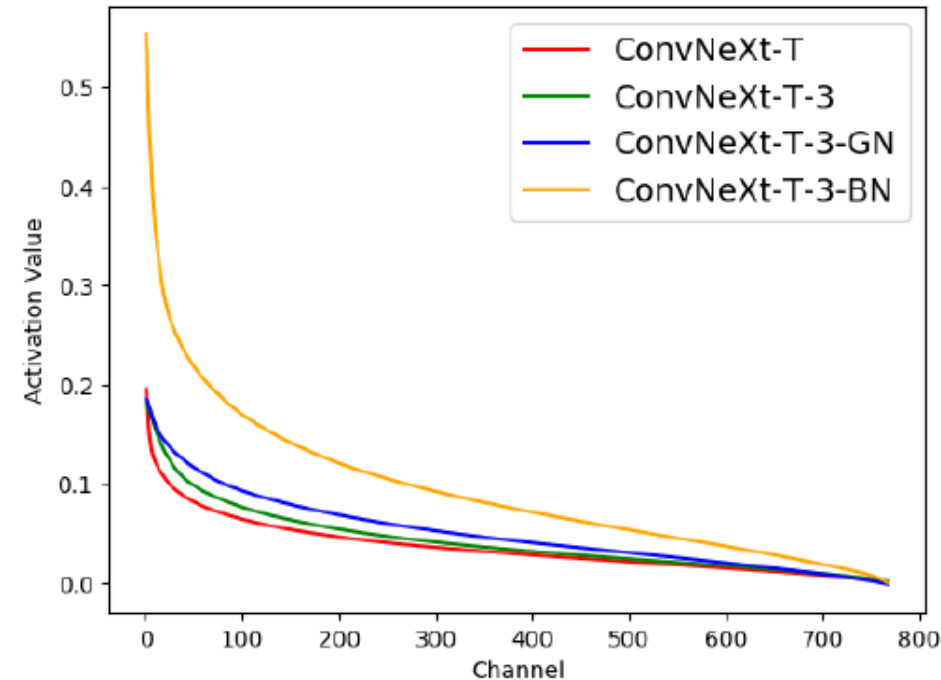


Figure 5. Sorted average values of the maximal activation in each image for each channel in the last block for ConvNeXt-T variants

Batch Normalization leads to a few channels with very high activations which tends to lead to making decisions with a few important features